

NYSC Staff Attendance — Staff Frontend

Project guide

Contents

1. 1. What this is
2. 2. How it's built
3. 3. The routes
4. 4. The rules the code enforces
5. 5. Decisions and why
6. 6. Running it on your phone
7. 7. Testing without a backend
8. 8. Connecting the backend
9. 9. What is not done
10. 10. Working with the admin dashboard

Project guide for the staff-facing mobile web application.

Written for a developer who knows React but has not seen this codebase. It covers what the product is, how the code is arranged, the rules it enforces, the decisions behind it, how to run it on a phone, and what the backend team needs to do to connect it.

1. What this is

NYSC offices across Nigeria record staff attendance on paper registers. Paper can be signed for someone else, lost, or damaged, and a plain "I am here" button on a website can be pressed from anywhere.

This application replaces that. Every office displays one permanent printed QR code. A staff member scans it with their own registered phone, and attendance is recorded only if three things are true at that moment: the account is active, the phone is the one bound to that account, and the person is inside the office geofence.

Who uses the system

Role	What they use
Staff	This app. A phone browser: register, log in, scan the office code
Office Admin	The admin dashboard (separate work, not in this guide)
Superuser	The same dashboard, across all offices

This repository currently contains the staff side only.

A staff member's day

They arrive at the office and point their phone camera at the printed code on the wall. The phone opens the attendance page. They tap **Sign in**, the phone asks for location, and the sign-in is recorded with a server timestamp. At the end of the day they scan again and tap **Sign out**. That calendar day, in Africa/Lagos time, is then complete and further scans are refused.

Before any of that, an admin adds them to the office staff list. The staff member then looks up their own staff ID in the app, confirms the record is theirs, sets a password, and verifies their email. The first phone they use is permanently bound to the account.

2. How it's built

Stack

Framework	Next.js 16.3.2, App Router
React	19.2.8
Language	TypeScript 5, strict mode
Styling	Tailwind CSS v4 — no config file, theme lives in <code>app/globals.css</code>
Components	shadcn v4 built on Base UI (not Radix)
HTTP	axios (real client stubbed, not yet wired)
Import alias	<code>@/*</code> maps to the repository root

The layered flow

Data moves in one direction. Each layer knows only about the one below it.

```
types/      The contract. What every record looks like.
|          No logic, no runtime code.
v
lib/mock/   Fixtures shaped by those types.
|          A staff member, an office, two weeks of attendance.
v
lib/api/    One `api` object. Mock or real, chosen by an env flag.
|          Screens import this and nothing else.
v
app/        Screens. They do not know where data came from.
```

Why it's arranged this way

The backend is not hosted yet, and the whole app has to be demonstrable before it is. The obvious approach — write `fetch` calls inside each screen and swap them later — means every screen has to be rewritten when the API arrives, and each one becomes a place where a mistake can hide.

Instead, `lib/api/types.ts` defines a single type called `ApiClient`. It lists every operation the staff app is permitted to perform. Two objects implement it: `lib/api/mock.ts` serves the fixtures, and `lib/api/real.ts` will call Django. `lib/api/index.ts` picks one:

```
const useMock = process.env.NEXT_PUBLIC_USE MOCK_API !== "false";
export const api: ApiClient = useMock ? mockApi : realApi;
```

Every screen imports the same thing:

```
import { api } from "@lib/api";
```

Connecting the real backend changes one file. No screen changes, because no screen ever knew which implementation it was talking to.

The flag defaults to mock, including when the variable is missing entirely. A fresh clone runs with no configuration and no backend. The failure direction is also safe: forgetting the flag can never point demo traffic at a real server, because the real client is the one you have to opt into.

Nothing throws

Every method on `ApiClient` returns a **discriminated union** — a type that is one of several shapes, where a `kind` field tells you which:

```
type LoginResult =  
  | { kind: "success"; staff: StaffProfile }  
  | { kind: "invalid_credentials" }  
  | { kind: "wrong_device" }  
  | { kind: "account_banned" }  
  | { kind: "email_not_verified"; email: string }  
  | { kind: "rate_limited"; retryAfterSeconds: number | null }  
  | { kind: "offline" }  
  | { kind: "error"; message: string };
```

Two things follow from this.

Failures are values, not exceptions. A `try / catch` you forget to write fails silently at runtime. A union the compiler makes you unpack fails at build time, on the exact line.

Data rides on the branch it belongs to. `signed_in` carries an `AttendanceDay`; `outside_geofence` carries nothing, because nothing was recorded. Reading `.day` on the wrong branch is a compile error, so a screen cannot render "Signed in at —" from a timestamp that was never sent.

assertNever

Every switch over a union ends the same way (`lib/utils.ts`):

```
default:  
  return assertNever(outcome);
```

`never` is TypeScript's term for a value that cannot exist. If every case is handled, the compiler narrows the argument to `never` and this compiles. Add an eleventh outcome to the union later and every switch that forgot it fails the build, pointing at the file that needs updating.

For a screen with twelve outcomes, that is the difference between a bug caught at compile time and a blank panel at an office door.

3. The routes

Thirteen routes, plus one demo tool.

Path	Purpose	Access
/	Entry point. Explains the product, links to sign in and register	Public
/login	Staff sign in. Sends <code>device_id</code> with credentials	Public
/register	Three-step claim flow: look up staff ID, confirm identity, set details	Public
/verify-email	Serves both <code>?email=</code> (check your inbox) and <code>?token=</code> (confirm)	Public
/forgot-password	Request a reset link	Public
/reset-password	Set a new password from <code>?token=</code>	Public
/home	Today at a glance and the main action	Protected
/scan	The attendance screen. Reads <code>?t=</code>	Protected
/history	Own records and statistics	Protected
/profile	Own record and device binding	Protected
/profile/password	Change password while signed in	Protected
/blocked	Dead end for a banned account or wrong device. Reads <code>?reason=</code>	Public
404	<code>app/not-found.tsx</code>	Public
/dev/qr	Demo tool. Renders a scannable office poster	Public

Protected routes live in `app/(protected)/`. The brackets make it a **route group** — it organises files without appearing in the URL, so `/home` is still `/home`. Everything in that group gets the auth guard, the top app bar and the bottom navigation.

`/scan` deliberately sits outside that group. It gets the same guard but neither bar, because the sign-in button must be reachable without scrolling at 375px and the two bars would take 108 pixels of the fold between them.

/scan in detail

This is the core screen. Source: `app/scan/scan-view.tsx`.

The printed poster holds a URL like `https://host/scan?t=qr_fct_hq_9f2a7c`. The phone camera opens it directly, so this page is the destination, not a scanner. There is also an in-app camera for someone already inside the app (`components/qr-scanner.tsx`), which navigates to the same URL on success, so both entry points run one flow.

The order of operations is the product rule expressed as code:

1. **Resolve the token and load today.** Two requests in parallel. No location involved.
2. **Show the office and wait.** Nothing happens until a deliberate tap.
3. **Ask for position.** Only now does the browser prompt.
4. **Submit and render the verdict.**

A dead token, an inactive office, or a day already complete all stop at step 1, so nobody is asked for GPS they did not need to give.

The twelve outcomes

Outcome	What the user sees	Can retry
Signed in	Confirmation with the recorded time	—
Signed out	Confirmation, day complete	—
Already complete today	Both times, further scans refused	—
Outside the geofence	"Please go to your office to sign in." Nothing recorded	Yes
Wrong device	Not your registered phone; an admin must reset it	No
Account banned	Deactivated; speak to your office admin	No
Office inactive	Attendance is not being recorded here	No
Invalid or rotated token	The poster may have been reprinted	No
Rate limited	Wait and scan again	No
Offline	Nothing was recorded; reconnect	Yes
Location permission denied	Allow location in browser settings	No
Location timed out	Common indoors; move towards a window	Yes

Location denial has no retry button on purpose. Once a browser stores a denial, the permission prompt never reappears, so a retry button would do nothing. Sending the user to settings is the only honest option.

The first ten come from the backend (`AttendanceOutcome` in `types/scan.ts`). The last two happen on the device before any request is made (`LocationFailure`), which is why they are a separate type — "you denied location" and "the server says you are on the wrong phone" are different kinds of event.

4. The rules the code enforces

These come from the product description and the phased build plan. The code is arranged so they are hard to break by accident, not merely documented.

Rule	Why	Where it lives
GPS is requested only at attendance time, and only on a tap	Asking on page load trains people to deny it, and a stored denial never re-prompts. Registration and login never need it	<code>app/scan/scan-view.tsx</code> , <code>lib/geolocation.ts</code>
One phone per staff member	Stops a colleague signing in for someone else. Staff cannot rebind; only an admin can reset	<code>lib/device.ts</code> , sent on every login and attendance call
The backend owns the geofence	A value the phone can read is a value the phone's owner can change	<code>types/office.ts</code> has no coordinates and no radius, by design
The backend owns lateness and statistics	Two implementations of the same arithmetic drift apart, and the server's answer is what appears in admin reports	<code>AttendanceDay.status</code> and <code>minutesLate</code> arrive as data; nothing is computed locally
Timestamps come from the server, never the device clock	A <code>Date</code> is re-expressed in the phone's timezone, so 07:52 in Lagos would read 06:52 in London	<code>lib/format.ts</code> slices the ISO string rather than parsing it
One sign in and one sign out per calendar day, Africa/Lagos	The day boundary is a server decision	<code>TodayResult</code> returns the server's <code>date</code> alongside the record
Registration is a claim, not a sign-up	An admin creates the record first. Otherwise anyone could invent a staff member	<code>app/register/</code> — the form never lets a user set their own name, office or department

Device binding in practice

`lib/device.ts` generates a UUID on first use and stores it under `nysc.device_id` in `localStorage`. It is sent with every login and every attendance submission.

This is **not a security boundary on its own**. Local storage can be cleared, and clearing it is exactly what a lost phone looks like from the frontend. What actually enforces the rule is the backend refusing a mismatch. The frontend's job is to produce a stable value and hand it over.

The module returns `null` rather than throwing when storage is unavailable — private browsing and blocked site data both throw on access — so a screen can say "we could not identify this device" instead of crashing.

5. Decisions and why

The choices below were not obvious, and each had a reasonable alternative.

Mock data behind a typed client. The alternative was writing `fetch` calls in each screen and replacing them later. That would mean rewriting every screen when the API lands, with each one a place for a

mistake to hide. Defining `ApiClient` first cost a day and makes the swap a single file. It also meant the twelve `/scan` states could be built and reviewed months before any endpoint existed.

Discriminated unions instead of thrown errors. The conventional approach is `throw` plus `try / catch`. The problem is that nothing forces you to write the `catch`, and a missing one fails silently in production. A union forces every caller to handle every branch, and `assertNever` turns a forgotten case into a build failure. The cost is more verbose types; the benefit is that a screen cannot ship with an unhandled state.

Light mode only. Supporting both schemes means verifying contrast twice for every state. On this app the states include "you are outside the geofence" and "this is not your phone" — messages that must be unambiguous under sunlight, on a cheap screen, at an office door. One scheme means one set of contrast ratios to check. It is locked in three places: `color-scheme: light` on `:root`, explicit `html / body` colours, and `colorScheme: "light"` in the root layout's viewport export.

48px primary buttons, 44px minimum everywhere. 44px is the accepted accessibility floor. The main action is deliberately above it, at 48px, because staff tap it one-handed, standing, often hurried. The `x1` button size is reserved for the single primary action on a screen — if every control were 48px, nothing would read as the main one.

Semantic colour tokens, not literal colours. `--primary` is NYSC green, `--success` is a confirmed sign in or out, `--destructive` is a refusal, and `--warning` is Late or Incomplete, which are neither. Components reference the meaning, not the colour, so rebranding is one block in `app/globals.css` and "it worked" never changes shade because the brand did.

The in-app camera is secondary, not primary. An earlier design put a QR scanner at the centre of the app. That was wrong: the printed code holds a URL, and the phone's own camera opens it directly, which needs no permission from us and no decoding library. Building the app around a scanner would have meant camera permissions, camera error states, and a library — all for a worse experience. The in-app camera exists for the person already inside the app who taps "Scan to sign in", and it navigates to the same URL the poster would have opened, so there is one flow rather than two.

Inputs are 16px. iOS Safari zooms the whole page when a field below 16px receives focus, and the layout stays wrong afterwards. `text-base` on every input prevents it. This looks like a styling detail and is actually the single most common mobile form bug.

6. Running it on your phone

The whole app works on a laptop, but the scan flow only makes sense on a real phone. This section assumes nothing.

Why HTTPS is required

Browsers refuse `navigator.geolocation` on an insecure origin. `localhost` is treated as secure, so a laptop works over plain HTTP — but `http://192.168.x.x` is **not** secure, so on a phone the location step fails with "Location is not available" no matter what permissions you grant.

The camera has the same restriction.

Step 1 — find your machine's address

```
ipconfig
```

Look for the IPv4 address of your active adapter, for example `192.168.123.146`. Both devices must be on the same network. A phone hotspot with the laptop joined to it works well; corporate wifi often blocks device-to-device traffic.

If your address differs from the one already listed, add it to `allowedDevOrigins` in `next.config.ts`. Next blocks cross-origin requests to dev assets by default, which breaks hot reload on a phone.

Step 2 — start the HTTPS dev server

```
yarn dev --experimental-https -H 0.0.0.0
```

The first run downloads **mkcert**, a small tool that creates a local certificate authority on your machine and issues a certificate for the dev server. Approve the operating-system prompt when it appears — it is asking for permission to trust that authority locally. The certificate is written to `certificates/`, which is git-ignored, and subsequent runs are instant.

`-H 0.0.0.0` makes the server listen on every network interface rather than just `localhost`, which is what allows the phone to reach it.

Step 3 — trust the certificate on the phone

Open `https://192.168.123.146:3000` on the phone. You will see **"Your connection is not private"**. This is expected: the certificate is self-signed and was issued for `localhost`, not for an IP address, so the browser flags both.

Tap **Advanced**, then **Proceed to ... (unsafe)**. It sticks for the session.

Do the same on the laptop.

Step 4 — if the phone still will not connect

Windows treats a hotspot as a public network and blocks inbound connections. Open an **administrator** terminal and run:

```
New-NetFirewallRule -DisplayName "Next dev 3000" -Direction Inbound -LocalPort 3000 -Protocol TCP -Action Allow
```

The tunnel alternative

If the network blocks device-to-device traffic entirely, or you want to demonstrate the app to someone who is not in the room, use a tunnel. It gives a real HTTPS URL with no certificate warnings and works from any network.

```
winget install cloudflare.cloudflare  
cloudflared tunnel --url http://localhost:3000
```

Run the ordinary `yarn dev` alongside it — the tunnel provides TLS, so `--experimental-https` is not needed. Cloudflare prints a `https://something.trycloudflare.com` address. The tunnel domains are already listed in `allowedDevOrigins`.

`npm localtunnel --port 3000` is the no-install option, but its free service returns "Bad Gateway" frequently.

Step 5 — a real walk-up test

1. On the **laptop**, open `https://192.168.123.146:3000/dev/qr`. This renders the office poster: the NYSC seal, the QR code, and instructions. Print it, or leave it on screen.
2. On the **phone**, open `https://192.168.123.146:3000/login` and sign in with the credentials in section 7.
3. Check `/home` reads "Not signed in yet".
4. Open the phone's **camera app** — not the browser — and point it at the code.
5. Tap the link. You should land on `/scan` showing the office name, still signed in.
6. Tap **Sign in**, allow location, and the sign-in is recorded.
7. Scan again for sign-out. A third scan reports "already done today".

The QR builds its URL from whatever origin the page was served on, so serve `/dev/qr` over the same HTTPS address you use on the phone. Opening it over HTTP would encode an HTTP URL and the location step would fail.

Checking the layout at 375px

In Chrome DevTools, click the device toolbar icon and select **iPhone SE (375px)**. That is the narrowest phone the app supports.

Worth checking on every screen: no horizontal scrolling, every tap target comfortably hittable, and the main action visible without scrolling on `/home` and `/scan`.

Also test dark mode rejection: press `Ctrl+Shift+P`, run **Show Rendering**, and set **prefers-color-scheme** to **dark**. Nothing should change anywhere. Password fields are where browsers auto-darken first.

7. Testing without a backend

Everything runs on fixtures in `lib/mock/`. No backend is required to review or demonstrate any part of the app.

Credentials

Purpose	Value
Working staff account	NYSC/FCT/0842
Password	Any 8 or more characters
Banned account	NYSC/FCT/0619
Already-claimed staff ID (registration)	NYSC/FCT/0619
Valid office token	qr_fct_hq_9f2a7c

Sign in with either the staff ID or the email (`c.ibrahim@nysc.gov.ng`). There is no stored password — `lib/api/mock.ts` accepts any string of 8 or more characters for a matching account.

Forcing login failures

Password	Result
<code>wrongdevice</code>	Wrong device
<code>unverified</code>	Email not verified
<code>offline</code>	Offline
Fewer than 8 characters	Invalid credentials

Forcing scan outcomes

Some outcomes cannot be produced by driving the app normally — there is no way to stand outside a fictional geofence. Append a query parameter to the `/scan` URL:

```
/scan?t=qr_fct_hq_9f2a7c&force=outside_geofence
```

`force` accepts any `AttendanceOutcome` kind: `signed_in`, `signed_out`, `already_complete`, `outside_geofence`, `wrong_device`, `account_banned`, `office_inactive`, `invalid_token`, `rate_limited`, `offline`, `error`.

`forceResolve` does the same for the token-resolution phase and accepts `invalid_token`, `rate_limited`, `offline`, `error`.

Location failures are reached through DevTools → Sensors → Location, or by blocking location for the site in the address bar.

This override lives entirely inside `lib/api/mock.ts` and reads `window.location` directly, so no screen and no shared type carries a hook for it. Nothing needs unpicking when the real client takes over.

Demo state

The mock stores its session under `nysc.mock.signedIn` and today's record under `nysc.mock.today`, both in `localStorage`. `localStorage` rather than `sessionStorage` because scanning the printed code opens a **new tab**, and `sessionStorage` is scoped to one tab — the session would be invisible and the user would be bounced to login at exactly the wrong moment.

Signing out clears both and resets the demo.

`/dev/qr` is a demo tool. In the real system the admin dashboard generates the poster from the `scan_url` the backend stores. Delete the route once that exists.

8. Connecting the backend

What changes

One file: `lib/api/real.ts`. Every method already has the correct signature and returns the correct union. Wiring one means writing the request and mapping the response onto that union.

Then set in `.env.local`:

```
NEXT_PUBLIC_USE MOCK_API=false
NEXT_PUBLIC_API_BASE_URL=https://your-api-host
```

What does not change

No screen, no type in `types/`, and no component. Screens import `api` and know nothing about the implementation behind it.

Endpoints the frontend expects

`ApiClient` (`lib/api/types.ts`) is the authoritative list. Paths marked **confirmed** appear in the phased build plan; the rest are proposals that need agreeing.

Method + path	Status	Request	Returns	Called by
POST /api/staff/login/	Confirmed	staffIdOrEmail , password , deviceId	Staff profile, or a typed failure	/login
POST /api/staff/register/	Confirmed	staffId , email , phone , password , deviceId	Success plus the email	/register step 3
GET /api/attendance/scan/? t=	Confirmed	Token in query	Office summary. No geofence check	/scan on load
POST /api/attendance/check/	Confirmed	token , deviceId , latitude , longitude , accuracy	Sign in, sign out, or a refusal	/scan on tap
GET staff lookup	Proposed	staffId	Name, office, whether already claimed	/register step 1
POST verify email	Proposed	token	Verified, expired, invalid, already done	/verify-email
POST resend verification	Proposed	email	Sent	/verify-email , /register
POST request password reset	Proposed	email	Always "sent" — see note	/forgot-password
POST reset password	Proposed	token , password	Success or a token failure	/reset-password
POST change password	Proposed	currentPassword , newPassword	Success or wrong- password	/profile/password
GET current profile	Proposed	—	Staff profile, or null	Auth guard on every protected route
GET today	Proposed	—	Server's date plus today's record or null	/home , /scan
GET history	Proposed	period , optional from / to	Day list plus statistics	/history

Method + path	Status	Request	Returns	Called by
POST logout	Proposed	—	—	/profile

Note on password reset: the response must be identical whether or not the address exists. Confirming which addresses are registered would turn the form into a way to discover who works at NYSC. The frontend has no branch for "no such account" and should not be given one.

When field names do not match

Django conventionally returns `snake_case`; these types use `camelCase`. Do not rename the types and do not touch the screens.

Map in one place — inside `lib/api/real.ts`, where each method already converts an HTTP response into a union. A small function per resource, for example `toAttendanceDay(raw)`, keeps every naming difference in a single file. If the backend later renames a field, one function changes.

The same applies to shapes that differ, such as a flat `department_name` where the frontend expects a nested `Department`.

Auth token

Not yet implemented — the mock has no token. When login is wired:

- Store the token via a small module alongside `lib/device.ts`, not scattered through screens.
- Attach it in a single axios request interceptor on the `client` instance already created at the top of `lib/api/real.ts`.
- On a `401`, clear the token and return `{ kind: "error" }` or a dedicated `session_expired` branch added to the relevant unions. Do **not** throw — the no-throw contract is what keeps screens exhaustive. The auth guard in `components/auth-provider.tsx` already redirects to `/login` when `getProfile()` returns null, so clearing the token is enough to recover.
- `device_id` is separate from the auth token and is sent on login and attendance regardless.

Questions to settle first

Three of these change type definitions, so answer them before writing code.

1. **Attendance status values on the wire.** The PRD names Present, Late, Absent, Checked Out and Incomplete in prose. The frontend assumes `present`, `late`, `absent`, `checked_out`, `incomplete`. Is the set closed?
2. **Is resumption time and grace period per office or global?** The build plan describes one global settings record holding radius and timezone and never mentions resumption time. The PRD says "configurable by the administrator" without saying at what level. `types/settings.ts` assumes global and is marked unconfirmed.
3. **Exact fields on an attendance record**, and which may be null. In particular, is `minutesLate` returned at all, or only the status?
4. Is there a single staff statistics endpoint covering days present, days absent, late arrivals, early departures, percentage, average arrival and departure times, and consecutive days — or must the

frontend assemble it?

5. Is department a nested object with an id, or a flat string? The frontend assumes nested, because department is filterable on the admin side.
6. Timestamp format: ISO 8601 with a UTC offset, or local time with none?
7. Is attendance percentage 0–100 or 0–1? Are average times "HH:mm" strings or minutes since midnight?

Two assumptions the frontend currently makes, both documented in `lib/mock/attendance.ts`: a day that is both late and completed keeps `status: "late"` rather than becoming `checked_out`; and `minutesLate` is measured from the resumption time, not from the end of the grace period.

Integration checklist

1. Answer questions 1–3 and adjust `types/` if needed.
2. Add the auth token module and the axios interceptor.
3. Wire `auth.login`, `auth.getProfile` and `auth.logout` first — nothing protected works without them.
4. Wire `attendance.resolveToken` and `attendance.submit`. Verify all twelve outcomes map onto real responses; the backend must distinguish wrong device from banned account from outside geofence, not return a generic 400.
5. Wire `attendance.today` and `attendance.history`.
6. Wire registration and password flows.
7. Set `NEXT_PUBLIC_USE MOCK_API=false` and walk the full flow on a phone.
8. Keep `lib/mock/` — it is how the app is demonstrated and how screens are developed without a running backend.

9. What is not done

- **No live endpoints.** `lib/api/real.ts` is a stub returning "not connected".
- **No auth token handling.** See section 8.
- `/history` period tabs all return the same fixture; the mock ignores the `period` argument.
- **No custom date range** on `/history`. `HistoryPeriod` includes "custom" but it needs a date picker.
- `/scan`, `/history` and `/profile` are functionally complete but have not had the visual pass that `/home` and `/login` have.
- `/register`, `/verify-email`, `/forgot-password`, `/reset-password` and `/blocked` do not yet carry the brand header.
- **No automated tests.** The mock scenario system makes manual verification fast, but there is no test suite.
- **Biometrics** are a future phase, deliberately not designed for and not blocked.

Worth doing next, in order: auth token handling, a test suite covering the twelve scan outcomes, the remaining visual pass, then the custom date range.

10. Working with the admin dashboard

The admin dashboard is separate work in the same repository.

Suggested split: admin under `/admin/`, staff at the root. The reason is the QR posters. They are printed once and stay on office walls, so whatever path `/scan` lives at is baked into physical paper. `/scan?t=...` is the URL that should be there, and changing it later means reprinting nationwide. Staff are also the far larger group, on phones; admins are a handful of people on desktops who can absorb a prefix.

`/login` is the real collision. Staff login calls `/api/staff/login/` with a `device_id` and is refused on the wrong phone. Admin login calls `/api/auth/login/` with no device binding at all. They cannot be one page — suggest `/login` for staff and `/admin/login` for admin.

Shared and worth keeping shared

- `types/` — `Office`, `Department`, `AttendanceDay`, `AttendanceStatus` and `AttendanceSettings` are the same records both halves render. One definition means one file changes when the backend answers the open questions. `StaffProfile` is staff-side; admin will need a fuller staff type carrying device and ban state.
- `components/ui/` — an `xl` size was **added** to `Button`; nothing existing was modified. Please extend the same way rather than changing defaults. `components/ui/input.tsx` is tuned for mobile (48px tall, 16px text). If that is wrong for dense desktop tables, add a size variant rather than lowering it.

One decision needs agreement: dark mode. The light lock described in section 5 is currently application-wide. If the dashboard wants dark mode, the lock moves off `:root` onto a wrapper around the staff routes and `colorScheme` comes out of the root layout. That is cheap now and considerably more expensive once both halves are built.